Agentic Graph Reasoning

Autonomous Knowledge Graph Navigation for Fact Verification and Hallucination Mitigation in Large Language Models

Md. Sakif Khan 0421052099

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

Supervisor: Dr. Sadia Sharmin, Associate Professor

Pre-defense

The problem

Language models answer factual questions fluently, and just as fluently when they do not hold the fact.

In our own no-retrieval control, on WebQSP:

- 661 entities asserted
- 179 of them do not exist in the knowledge graph
- that is 27.1%, stated with no hedge

The gap

Fluency and factuality are produced by the same mechanism, so the output gives the reader no signal about which one they are getting.

A system that is confidently wrong is worse than one that says it does not know.

The failure is not rare, and it is not visible in the answer.

Why the model does this

A hallucination is fluent, well formed, and backed by no source the model can point to — not a bug that escaped testing, but a consequence of how these models are built.

Three places it comes from

1. The training data — errors and thin coverage, reproduced faithfully
2. The model — knowledge is spread across the weights with no index, so nothing separates a memorised fact from a plausible continuation
3. The prompt — what we put in the context window

Which can we act on?

Only the third. The first two need a curated corpus or a retrained model.

The third is a systems problem — and that is this thesis.

Multi-hop compounds it: a wrong first hop rarely surfaces, so the chain proceeds from a false premise.

Where existing approaches stop

Approach	What it does	Where it stops
Parametric	Answers from weights	No external evidence at all
Vector RAG	Retrieves text once, then reasons	Retrieval happens *before* reasoning begins
Static GraphRAG	Retrieves a fixed neighbourhood	Radius fixed before the question is understood
Agentic navigation	Interleaves retrieval and reasoning	Emits whatever the last generation call produces

Nobody checks what the answer asserts *before* it is delivered.

Research questions

RQ1 Does agentic navigation improve multi-hop factual accuracy — and does the advantage grow with hop count?

RQ2 Given a system that already navigates the graph, what does a claim-level check against traversed triples contribute?

RQ3 Which components earn their cost, in accuracy and in tokens?

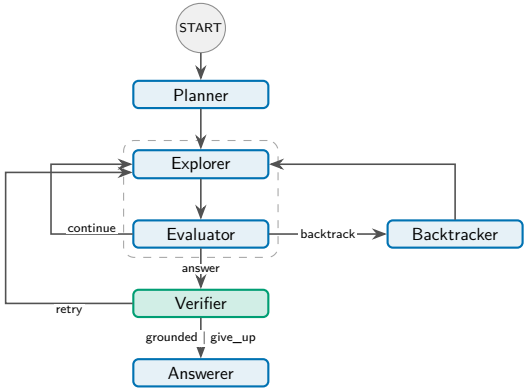
RQ2 is posed as *what does it contribute*, not *does it reduce hallucination*. The answer has several parts, and a yes/no framing would hide them.

AGR: an explicit state machine

Six nodes; the model is consulted only at decision points.

- **Planner** — sub-objectives
- **Explorer** — expands the frontier
- **Evaluator** — objective met?
- **Backtracker** — undo + ban list
- **Verifier** — the contribution
- **Answerer** — emits what survived

Three cycles — *continue*, *backtrack*, *retry* — all bounded by budgets rather than by model behaviour.



Constrained tools, not free-form queries

Operation	Returns	Why it is bounded
`get_relations`	Candidate relations	≤ 300 offered to the scorer
`get_neighbors`	Neighbours of an entity	≤ 200 per expansion
`verify_connection`	Adjacency in the full graph	Used only by the verifier
`search_entity`	Surface form $\rightarrow$ node	Three-stage resolver

The agent never writes Cypher; every operation is logged.

Determinism in the tools is what makes the verification layer possible.

The Structural Verification Layer

Before anything is emitted:

1. Draft answer is split into atomic claims
2. Each claim is checked against the triples actually traversed
3. Claims grounded by the traversal keep those triples as evidence
4. Unsupported claims trigger targeted re-exploration, or are dropped

The system therefore **hedges rather than asserts**, and carries its evidence with the answer — from one route of three, and only inside the run.

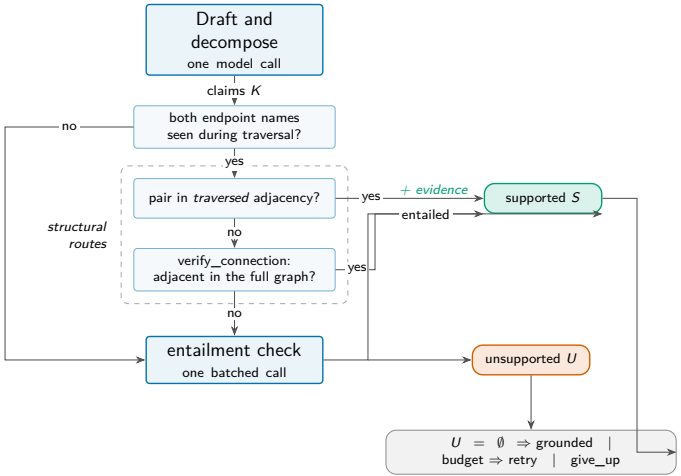
What *structural* means

The check is against the graph the agent walked, not the model's judgement of its own output. But it tests adjacency: whether an edge joins the claim's endpoints, not whether it is the edge the claim *names*, and not its direction. A *mother* claim passes on a *child* edge — only route three reads the relation.

The other honest limit

A claim can be *true* and still be the wrong answer. Structural grounding cannot catch that — see the echo attractor later.

One claim, three routes



Only the third route costs a model call. The first two are pure graph lookups — and neither of them reads the relation.

The environment and the question sets

Freebase-derived graph

Property	Value
Entities	2,592,892
Triples	8,309,194
Distinct relations	7,058
Import time	36.4 s

Certified question sets

	WebQSP	CWQ
Questions	400	400
Gold (median)	1.5	1.0
Reachable	97.0%	99.2%
Multi-hop	132	260

Reachability is the *ceiling* on every Hits@1 reported: a miss is the system's, not the environment's.

800 **questions, with reachability certified per question so the accuracy ceiling is known.**

Making the comparison fair

Five systems, one frozen backbone, one budget:

- No-retrieval (parametric control)
- Vector RAG
- Static GraphRAG
- Think-on-Graph (agentic)
- **AGR**

Same model, temperature 0, same 25-call cap, same questions, same graph.

What this buys

Differences are attributable to **architecture**, not to model capacity or spend.

What is *not* held equal

ToG prunes to 40/20 candidates per step, AGR to 300/200 — narrower is cheaper, so it cannot explain ToG's clipping, but its unclipped score is a lower bound.

Why a parametric control

These benchmarks predate current models and may be partly memorised. Without it we could not separate retrieval from recall.

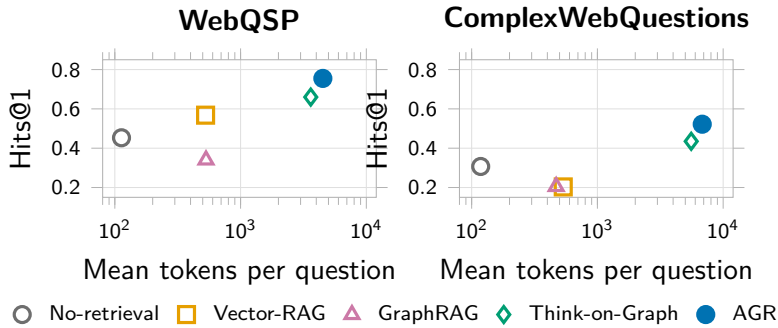
Main results

	WebQSP		ComplexWebQuestions		Cost / question	
System	**Hits@1**	**F1**	**Hits@1**	**F1**	**Tokens**	**Calls**
No-retrieval	0.453	0.305	0.307	0.279	113	1.0
Vector RAG	0.568	0.432	0.203	0.168	527	1.0
Static GraphRAG	0.340	0.262	0.205	0.183	531	1.0
Think-on-Graph	0.660	0.477	0.435	0.378	3,615	12.8
AGR	**0.755**	**0.642**	**0.522**	**0.469**	4,511	**6.2**

Cost is WebQSP. CWQ: AGR 6,818 tokens, 8.9 calls; Think-on-Graph 5,572, 18.2.

Ahead of every baseline on both datasets, at half the model calls of the other agentic system.

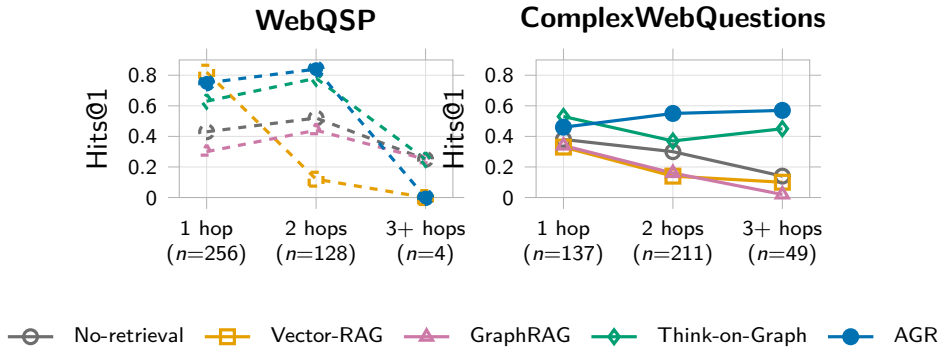
Accuracy against cost



On *tokens* Think-on-Graph is **not** dominated — it buys lower accuracy at a lower token price. On *calls*, which the budget meters, it is.

The frontier depends on which cost you charge.

RQ1: accuracy against hop count



On CWQ, AGR goes 0.46 → 0.55 → 0.57 — the **only** system that ends above where it started. Three of the other four decay monotonically.

RQ1 is answered by a shape, not a difference of means.

The caveat I want to raise myself

	WebQSP		ComplexWebQuestions	
Hits@1 on...	**ToG**	**AGR**	**ToG**	**AGR**
questions ToG *finishes*	**0.852**	0.788	**0.629**	0.607
questions ToG is *clipped* on	0.197	**0.675**	0.188	**0.415**
clip rate	29.2% (117/400)		44.0% (176/400)	

Where Think-on-Graph finishes inside the shared 25-call cap, it is **ahead of AGR on both datasets**. The entire aggregate margin comes from the questions it cannot finish.

The claim is efficiency under a fixed budget — not that AGR reasons better per step.

RQ2: does anything ungrounded get asserted?

System	Entities asserted	Ungrounded	Rate
No-retrieval	1,001	221	22.1%
Vector RAG	1,329	45	3.4%
Static GraphRAG	1,024	8	0.8%
Think-on-Graph	1,265	0	**0.0%**
AGR	1,709	0	**0.0%**

Both datasets pooled. AGR asserts no entity that does not exist in the graph, across 1,709 assertions.

But Think-on-Graph reaches 0.0% too — so this is a property of *graph navigation*, not of the verification layer.

RQ2: so what does verification actually contribute?

What it does *not* do

Removing it changes accuracy by an amount we cannot detect: $p = 1.0$ on *both* datasets. It does not earn its place on Hits@1 or F1.

What does *not* survive the run

The log keeps the count of supporting triples and discards the list. *Auditable in the run, not from the record.*

What it does do

- Withholds what it cannot ground, as a hedge: removing the layer drops hedging 23.2% $\rightarrow$ 20.2% on CWQ, 8.5% $\rightarrow$ 8.0% on WebQSP
- Attaches supporting triples — from one route of three; verify_connection and entailment attach none
- Pairs answer with evidence, in the run

This is a negative result reported as one. The layer's case rests on auditability, and the thesis bounds that too rather than claiming an accuracy benefit the measurement does not support.

RQ3: what each component earns

	WebQSP		ComplexWebQuestions		
Condition	**F1**	p	**F1**	p	**Tokens (WQ)**
Full system	0.659	—	0.445	—	4,562
No planner	**0.742**	**0.006**	0.398	0.088	**3,159**
No backtracking	0.646	0.727	0.445	1.000	4,261
No verifier	0.663	1.000	0.436	1.000	4,460
Embedding-only scoring	0.643	0.664	0.437	0.481	3,413

Paired McNemar against the full system, stratified halves ($n = 200$ / 198).

Exactly one component shows a detectable accuracy effect — and the sign is backwards.

The result I did not expect

Removing the planner *improves* WebQSP:

- $+0.083$ F1, at $p = 0.006$
- -31% tokens
- $6.2 \rightarrow 4.0$ calls

On ComplexWebQuestions it trends the other way (-0.047 F1, $p = 0.088$) — not significant, but the opposite sign.

Why

WebQSP is predominantly one hop. Decomposing a one-hop question invents a second sub-objective that sends the frontier away from the answer.

What follows

Decomposition should be **gated on question structure**, not applied unconditionally. That is a design conclusion the measurement forced, not one it confirmed.

Every failure, read: the echo attractor

All 259 remaining failures were read and labelled by hand, both datasets pooled:

Category	n
Relation selection	65
Composite claim	47
Knowledge-graph gap	44
Decomposition error	38
Answer selection	15
Echo attractor	13

Totals. Wrong and hedge are *never pooled* in the thesis, and the shape flips: composite claim is 1 on WebQSP against 46 on CWQ. Split census: backup page 4.

The characteristic error of a graph navigator is not invention. It is the **echo attractor**: a real, grounded entity one hop from the answer.

Why it matters

Any grounding check *passes* it — because it is true. Verification cannot catch it by construction.

Different systems fall into it together, so no evaluation treating them as independent can see it — and a policy of rescoring on majority agreement turns it into apparent correctness.

The benchmark was wrong 57 times

The same cross-system agreement is also a detector: consensus flagged 105 questions, adjudication confirmed 41, and the gap is the attractor above rather than label noise.

- 41 excluded before the census
- 17 found inside the census
- 1 counted in both

$\Rightarrow$ 57 **distinct questions**

Why report it

Reported defect rates for these two standard benchmarks are rare and usually anecdotal. This is a counted rate with a documented adjudication procedure and per-question provenance.

A contribution that outlives this particular system.

Contributions, and what I would not claim

Contributions

1. AGR and its Structural Verification Layer
2. A component-level ablation of four mechanisms
3. Stratum-dependent decomposition — planning hurts
4. The *echo attractor*, named
5. Benchmark-defect rates for WebQSP and CWQ (57)
6. Pre-specified evaluation thresholds

Limitations I state plainly

- The verifier logs only what it rejects: wrongful acceptance is unmeasured, and the evidence is not persisted
- No detectable accuracy gain — and at $n \approx 200$ that is "not detected", not "none"
- Zero ungrounded assertion is navigation, not the layer
- One environment, one backbone, one annotator
- ToG leads where it finishes, from a narrower candidate set: 40/20 vs 300/200

Thank you

Questions